

TJSC 2026 - Analista de Sistemas

Engenharia de Software e Desenvolvimento

Aula 1 - Engenharia de requisitos, análise e projeto de sistemas

Material autoral orientado à FGV

Aula estruturada para cobrir requisitos, análise e projeto com foco em acerto de questão objetiva. As questões são inéditas, simulam padrões de formulação observados em cadernos oficiais da FGV, mas não reproduzem questões reais.

Perfil considerado: desenvolvedor Java/Quarkus e Angular/TypeScript, com vivência em APIs, integração, documentação técnica, Git, esteiras corporativas e sistemas internos. Estratégia: converter experiência prática em domínio de terminologia, etapas, artefatos e pegadinhas de prova.

Fontes de alinhamento: edital TJSC/FGV 2026; cadernos oficiais recentes da FGV para TI; ISO/IEC/IEEE 29148; BABOK Guide; OMG UML; literatura clássica de engenharia de software.

Como usar esta aula

Esta aula cobre a parte de maior ROI inicial em Engenharia de Software: entender o que é requisito, como ele é descoberto, analisado, documentado, validado e transformado em desenho de solução. A FGV costuma cobrar tanto definição conceitual quanto interpretação de cenários longos: um órgão público, uma equipe, uma falha de escopo, um requisito não funcional mal formulado, um artefato confundido com outro ou uma decisão de projeto travestida de requisito.

Meta prática da aula

Ao final, você deve conseguir distinguir levantamento, elicitación, análise, especificação, validação e gerência de requisitos; separar requisito funcional, não funcional, requisito de domínio, regra de negócio e restrição de projeto; diferenciar análise e projeto; e responder questões com cenários corporativos no estilo FGV.

Mapa de ROI para o TJSC

Bloco	O que a FGV tende a explorar	Ganho esperado
Engenharia de requisitos	Etapas, técnicas, tipos de requisitos, boas características, validação, rastreabilidade e mudanças.	Alto: base para várias questões de Engenharia e Arquitetura.
Análise de sistemas	Modelagem do domínio, processos, regras, atores, casos de uso, dados e entendimento do problema.	Alto: evita confundir análise com projeto/codificação.
Projeto de sistemas	Transformação do problema em solução: arquitetura, componentes, interfaces, banco, camadas e decisões técnicas.	Médio/alto: conecta com arquitetura, APIs e padrões.
Prática x prova	Na vida real os limites se misturam; na prova, a banca separa fases e conceitos com precisão.	Alto: principal fonte de erro para quem já trabalha com desenvolvimento.

Diagnóstico do seu histórico

Seu histórico anterior indica desempenho geral bom, mas Engenharia de Software aparece como disciplina com margem clara de crescimento. Em especial, testes, métricas, orientação a objetos e análise de pontos de função apareceram como pontos frágeis. Embora esses últimos temas sejam aprofundados em aulas futuras, a base de requisitos e análise precisa estar firme, porque requisitos mal entendidos contaminam testes, métricas, modelagem e arquitetura.

1. Engenharia de requisitos: visão de prova

Engenharia de requisitos é o conjunto de atividades voltadas a descobrir, analisar, especificar, validar e gerenciar necessidades e restrições de um sistema. Em prova, não trate requisito como sinônimo pobre de “pedido do usuário”. Requisito é uma necessidade, capacidade, condição ou restrição que deve ser satisfeita por uma solução.

A norma ISO/IEC/IEEE 29148 trata engenharia de requisitos como processos e produtos relacionados à definição de requisitos de sistemas e software ao longo do ciclo de vida. Para a prova, isso importa porque requisito não nasce pronto: ele é construído por interação com stakeholders, análise do domínio, documentação e validação.

Tradução para sua rotina de dev

Quando alguém no banco diz “preciso de uma tela para aprovar pagamento”, isso ainda é uma solução candidata. A necessidade real pode ser: registrar decisão de aprovação, garantir trilha de auditoria, respeitar alçadas, impedir duplicidade e integrar com APIs internas. A prova cobra essa separação entre necessidade, regra, restrição e solução técnica.

1.1 Etapas clássicas

Etapas	Ideia central	Exemplo em sistema corporativo
Elicitação	Descobrir necessidades, expectativas, restrições e problemas junto a stakeholders e fontes do domínio.	Entrevistar usuários da unidade, observar fluxo de aprovação, analisar normativos e sistemas legados.
Análise	Resolver conflitos, priorizar, decompor,	Separar regra de alçada, regra de convênio,

	modelar, entender viabilidade e consistência.	requisito de auditoria e requisito de desempenho.
Especificação	Registrar requisitos em formato compreensível, rastreável e verificável.	Documento de requisitos, backlog, casos de uso, histórias, critérios de aceite, modelos de processo.
Validação	Confirmar com stakeholders que os requisitos documentados refletem a necessidade real.	Revisão com área de negócio, protótipo navegável, walkthrough, aceite dos critérios.
Gerência	Controlar mudanças, versões, rastreabilidade, priorização e baseline.	Matriz requisito -> história -> commit -> teste -> release; gestão de mudança de escopo.

Pegadinha FGV

A banca pode trocar “elicit” por “levantar”. Levantamento costuma ser usado informalmente, mas elicitação é termo técnico mais amplo: inclui entrevistas, observação, workshops, análise documental, prototipação, questionários e outras técnicas para revelar necessidades explícitas e tácitas.

1.2 Requisito funcional, não funcional e restrições

A classificação mais cobrada separa o que o sistema faz, como ele deve se comportar e quais limites condicionam a solução.

Tipo	Definição de prova	Exemplo
Requisito funcional	Serviço, comportamento ou função que o sistema deve fornecer.	O sistema deve permitir que o servidor consulte processos por número, parte ou CPF/CNPJ.
Requisito não funcional	Qualidade, propriedade ou restrição de qualidade do sistema ou do processo.	A consulta deve responder em até 2 segundos para 95% das requisições em horário comercial.
Regra de negócio	Regra do domínio, da organização ou da legislação que orienta o comportamento do sistema.	A aprovação acima de determinado valor exige dupla autorização.
Requisito de domínio	Condição derivada do domínio de aplicação, muitas vezes técnica ou normativa.	O sistema deve preservar metadados arquivísticos conforme política institucional.
Restrição de projeto	Limitação imposta à solução, tecnologia, ambiente, padrão ou processo.	A API deve ser implementada em Java 17+ e exposta em padrão REST.

Requisitos não funcionais incluem desempenho, disponibilidade, segurança, usabilidade, acessibilidade, interoperabilidade, auditabilidade, confiabilidade, escalabilidade, portabilidade e manutenibilidade. Para TJSC, tenha atenção especial a segurança, proteção de dados, auditoria, gestão documental digital, preservação e integração entre sistemas.

1.3 Boas características de requisitos

Um requisito de boa qualidade deve ser escrito de modo a reduzir ambiguidade e permitir verificação. A FGV gosta de cobrar esse ponto por alternativas com palavras como “subjetivo”, “genérico”, “flexível”, “sem necessidade de teste” ou “completo apenas na visão do desenvolvedor”.

Característica	Como memorizar	Exemplo bom
Necessário	Não é luxo nem solução sem vínculo com objetivo.	O sistema deve registrar o responsável por cada aprovação.
Não ambíguo	Uma interpretação plausível.	Até 2 segundos é melhor que “rápido”.
Completo	Contém condições, exceções e critérios relevantes.	Inclui caminho alternativo quando a integração externa falhar.
Consistente	Não contradiz outros requisitos.	Não exige “aprovar automaticamente” e “exigir dupla aprovação” no mesmo caso.
Verificável/testável	Permite checar por teste, inspeção, análise ou demonstração.	Registrar log com usuário, data, hora e operação.
Rastreável	Pode ser ligado à origem, implementação e teste.	REQ-SEG-04 -> US-18 -> Teste E2E-07.
Viável	Cabe nas restrições técnicas, legais,	Evita prometer disponibilidade irrereal sem

	orçamentárias e temporais.	arquitetura compatível.
--	----------------------------	-------------------------

Hack de prova

Quando a alternativa diz que requisito não funcional “não precisa ser testável” ou que requisito bom pode ser “genérico para manter flexibilidade”, desconfie. Requisito bom tende a ser claro, verificável e rastreável.

2. Técnicas de elicitación

Elicitación é a atividade de descobrir requisitos a partir de pessoas, documentos, sistemas existentes e observação do trabalho real. A técnica ideal depende do tipo de conhecimento buscado. Em sistemas corporativos, muito requisito não está na cabeça de uma pessoa: está em normativo, planilha legada, exceção operacional, comportamento tácito do usuário e integração com sistemas externos.

Técnica	Quando usar	Pegadinha comum
Entrevista	Explorar necessidades com profundidade, obter contexto e motivação.	Não garante consenso; pode refletir visão parcial do entrevistado.
Workshop/JAD	Alinhar múltiplos stakeholders, resolver conflitos e construir consenso.	Exige facilitação; não é apenas reunião genérica.
Observação/etnografia	Entender trabalho real, exceções, atalhos e conhecimento tácito.	O usuário pode fazer diferente quando observado.
Questionário/survey	Coletar dados de muitos usuários com baixo custo.	Pouca profundidade; depende da qualidade das perguntas.
Análise documental	Extraír regras de normas, manuais, contratos, legislações e sistemas legados.	Documento pode estar desatualizado ou incompleto.
Prototipação	Reduzir ambiguidade de interface, fluxo e experiência esperada.	Protótipo não substitui requisito; pode induzir apego prematuro à solução.
Brainstorming	Gerar ideias e possibilidades rapidamente.	Não valida viabilidade nem prioridade por si só.

2.1 Stakeholders

Stakeholder é qualquer pessoa, grupo, órgão, sistema ou entidade que afeta, é afetado ou percebe ser afetado pela solução. Em prova, stakeholder não é apenas usuário final. Inclui área de negócio, TI, suporte, segurança, auditoria, jurídico, administração de dados, órgão regulador, equipe de operação, gestores, fornecedores e sistemas integrados.

Em um sistema do TJSC, por exemplo, podem ser stakeholders: servidor que usa o sistema, magistrado, área de tecnologia, corregedoria, equipe de segurança da informação, setor de gestão documental, CNJ quando há resolução aplicável, cidadão atendido indiretamente e sistemas externos integrados.

2.2 Elicitación em sistemas com APIs e integrações

Para sistemas integrados, a elicitação não pode parar na tela. É preciso descobrir contratos, dados obrigatórios, regras de idempotência, mensagens de erro, timeouts, autenticação, autorização, trilhas de auditoria, SLA - Service Level Agreement, e comportamento em falhas. Isso conecta Engenharia de Software com Arquitetura e Segurança.

Exemplo Quarkus/Angular

Uma história como “Como usuário, quero consultar pagamentos” é insuficiente se não houver critérios sobre filtros, paginação, autorização, logs, tratamento de erro da API, latência, campos sensíveis e mensagens ao usuário. A FGV pode explorar exatamente essa lacuna entre pedido funcional e requisitos completos.

3. Análise de requisitos

A análise organiza aquilo que foi descoberto. É nessa fase que a equipe identifica inconsistências, lacunas, duplicidades, conflitos, prioridades e dependências. Também é onde se separa necessidade de solução, problema de tecnologia e regra de negócio de decisão de implementação.

3.1 Atividades típicas

- **Classificar requisitos:** funcional, não funcional, regra de negócio, domínio, restrição, transição.
- **Modelar o domínio:** entidades, conceitos, eventos, estados, atores e relacionamentos.
- **Modelar processos:** fluxo atual e futuro, gargalos, exceções e handoffs entre áreas.
- **Negociar conflitos:** stakeholders diferentes podem ter objetivos incompatíveis.
- **Priorizar:** valor de negócio, risco, dependência, urgência, obrigação normativa e esforço.
- **Definir critérios de aceite:** condições verificáveis para considerar requisito atendido.
- **Avaliar viabilidade:** técnica, operacional, legal, econômica, temporal e organizacional.

3.2 Análise de negócios x análise de sistemas

A FGV pode tratar análise de negócios e análise de sistemas como atividades próximas, mas não idênticas. Análise de negócios olha problema, valor, processo, necessidade e mudança organizacional. Análise de sistemas traduz esse entendimento para requisitos, modelos e comportamentos de uma solução informatizada.

Aspecto	Análise de negócios	Análise de sistemas
Pergunta central	Qual problema/necessidade de negócio precisa ser resolvido?	Que comportamento e estrutura o sistema deve ter para atender à necessidade?
Foco	Valor, processo, stakeholders, objetivos, riscos e mudança.	Requisitos, modelos, dados, interfaces, regras e fronteiras do sistema.
Artefatos comuns	Business case, mapa de processos, canvas, matriz de stakeholders.	Casos de uso, histórias, especificação, modelos de dados, protótipos, diagramas.
Erro de prova	Achar que análise de negócios é programação.	Achar que análise de sistemas é desenhar tela ou escrever código.

3.3 Priorização

Priorização define ordem de entrega e foco de esforço. A técnica MoSCoW classifica requisitos em Must have, Should have, Could have e Won't have now. Em português de prova: obrigatório, importante, desejável e fora do escopo atual. Outras abordagens ponderam valor, risco, custo, dependência e urgência regulatória.

No setor público, "valor" não é apenas receita: pode ser conformidade normativa, redução de risco, transparência, continuidade do serviço, segurança, acessibilidade e eficiência administrativa.

4. Especificação de requisitos

Especificar é registrar requisitos de forma organizada. A especificação pode assumir formatos tradicionais, ágeis ou híbridos. A FGV não costuma premiar dogmatismo: documento formal, caso de uso, história de usuário, protótipo e modelo visual podem coexistir se forem úteis, consistentes e rastreáveis.

4.1 Artefatos de especificação

Artefato	Uso típico	Cuidado de prova
ERS/SRS - Especificação de Requisitos de Software	Documento estruturado com requisitos funcionais, não funcionais, interfaces, restrições e critérios.	Não é código nem desenho detalhado de solução.
Caso de uso	Descreve interação entre ator e sistema para atingir objetivo.	Ator não precisa ser pessoa; pode ser outro sistema.
História de usuário	Formato ágil: como [papel], quero [capacidade], para [benefício].	Não dispensa critérios de aceite nem conversas.
Crítérios de aceite	Condições objetivas para aceitar uma funcionalidade.	Devem ser verificáveis.
Protótipo	Representa interface ou fluxo para esclarecer entendimento.	Não substitui especificação de regras, integrações e restrições.
Modelo de domínio	Representa conceitos e relacionamentos relevantes.	Não é necessariamente o modelo físico do banco.

4.2 Caso de uso x história de usuário

Caso de uso é mais estruturado e detalha atores, pré-condições, fluxo principal, fluxos alternativos, pós-condições e exceções. História de usuário é mais leve e serve como unidade de conversa e planejamento em métodos ágeis. A FGV pode tentar afirmar que uma história de usuário substitui toda documentação; isso é exagero.

4.3 Critérios de aceite

Critérios de aceite conectam requisito e teste. Bons critérios reduzem subjetividade. Exemplo: “Dado que o usuário possui perfil de aprovador e existe uma solicitação pendente, quando ele confirmar a aprovação, então o sistema deve registrar usuário, data, hora, decisão e justificativa no log de auditoria”.

5. Validação, verificação e gerência de requisitos

5.1 Verificação x validação

Verificação pergunta se o artefato foi produzido corretamente: o requisito está claro, consistente, completo, não ambíguo e verificável? **Validação** pergunta se o requisito certo foi capturado: ele atende à necessidade real dos stakeholders?

Mnemônico

Verificação: “fiz certo o artefato?”

Validação: “fiz o artefato certo para a necessidade?”

Na prática: verificação encontra defeito de escrita; validação encontra desalinhamento com o problema real.

5.2 Técnicas de validação

- **Revisão/inspeção:** leitura sistemática de requisitos por pares e stakeholders.
- **Walkthrough:** percorrer cenários e fluxos para identificar lacunas.
- **Prototipação:** obter feedback sobre interface, fluxo e entendimento.
- **Casos de teste derivados de requisitos:** se não dá para testar, talvez o requisito esteja ruim.
- **Matriz de rastreabilidade:** verificar cobertura entre necessidade, requisito, implementação e teste.

5.3 Rastreabilidade

Rastreabilidade é a capacidade de acompanhar a origem, evolução e implementação de um requisito. Ela pode ser para trás, para frente e bidirecional. Em ambiente corporativo, rastreabilidade ajuda auditoria, manutenção, impacto de mudanças e cobertura de testes.

Tipo	Pergunta respondida
Para trás	De onde veio este requisito? Qual stakeholder, norma, incidente ou objetivo o originou?
Para frente	Onde este requisito foi implementado, testado e entregue?
Bidirecional	Consigo navegar da origem até o teste e do teste até a origem?

5.4 Baseline e controle de mudanças

Baseline é uma versão aprovada de um conjunto de requisitos, usada como referência para desenvolvimento, testes e controle de mudanças. Mudança de requisito não é proibida, mas deve ser analisada quanto a impacto, risco, custo, prioridade, prazo e dependências.

Em provas, desconfie de alternativas absolutas: “requisito não muda”, “mudança deve sempre ser rejeitada”, “mudança dispensa análise se vier do usuário”, “backlog dispensa controle”. Em métodos ágeis há adaptação, mas não ausência de gestão.

6. Análise e projeto de sistemas

A análise concentra-se no entendimento do problema; o projeto concentra-se na concepção da solução. Em equipes reais, essas fronteiras se sobrepõem; na prova, a banca gosta de separá-las.

Dimensão	Análise	Projeto
Pergunta	O que o sistema deve resolver e quais regras deve respeitar?	Como o sistema será estruturado para resolver isso?
Foco	Domínio, necessidades, requisitos, regras, fluxos, atores.	Arquitetura, componentes, interfaces, classes, banco, camadas, padrões.
Exemplo	"Apenas aprovadores podem autorizar pagamentos acima da alçada."	Usar serviço de autorização, camada de domínio, tabela de alçadas e interceptor de segurança.
Erro comum	Colocar solução técnica como se fosse requisito de negócio.	Projetar sem entender o problema e as restrições.

6.1 Projeto lógico x projeto físico

Projeto lógico descreve a solução em nível conceitual ou independente de detalhes físicos: módulos, responsabilidades, fluxos, entidades, interfaces. **Projeto físico** aproxima a solução da implementação: tecnologia, banco específico, estrutura de tabelas, deployment, infraestrutura, frameworks e configurações.

Em banco de dados, por exemplo, modelo lógico relacional não é a mesma coisa que script físico Oracle/PostgreSQL. Em software, uma arquitetura em camadas não é a mesma coisa que configurar o pom.xml, o Dockerfile e o pipeline.

6.2 Requisitos não funcionais dirigem projeto

Muitas decisões arquiteturais não vêm de requisitos funcionais, mas de requisitos não funcionais. Segurança pode exigir autenticação forte, autorização por perfil, criptografia e logs. Disponibilidade pode exigir redundância. Desempenho pode exigir cache, índices ou processamento assíncrono. Auditabilidade pode exigir trilhas imutáveis.

Conexão com APIs

Uma API REST que "consulta processos" atende a uma função. Mas autenticação, autorização, idempotência, observabilidade, versionamento, paginação, limites de taxa e logs são decisões e requisitos de qualidade que afetam projeto e arquitetura.

6.3 CASE - Computer-Aided Software Engineering

Ferramentas CASE - Computer-Aided Software Engineering - apoiam atividades de engenharia de software, como modelagem, documentação, geração de artefatos, rastreabilidade e gestão de requisitos. Elas não substituem análise crítica nem garantem qualidade por si só. FGV pode cobrar exatamente isso: ferramenta apoia, mas não decide o projeto pelo analista.

7. Prática real x cobrança FGV

Na prática corporativa	Na prova FGV
O time chama tudo de "demanda", "história" ou "card".	A banca separa necessidade, requisito, regra de negócio, restrição, critério de aceite e solução.
Analista, dev e PO discutem solução ao mesmo tempo.	A banca diferencia elicitação, análise, especificação, validação e projeto.
Documento e backlog coexistem informalmente.	A banca cobra artefatos por finalidade: caso de uso, ERS/SRS, protótipo, matriz de rastreabilidade.
Mudança de escopo é negociada em reunião.	A banca cobra baseline, controle de mudança e análise de impacto.
Requisito não funcional aparece tarde, quando dá problema.	A banca trata NFR como requisito essencial desde o início.
Dev tende a ir para solução: endpoint, tabela, serviço.	A banca pune confusão entre requisito e desenho técnico.

8. Siglas e termos para memorizar

Sigla/termo	Expansão	Uso em prova
SRS	Software Requirements Specification	Especificação de Requisitos de Software.
ERS	Especificação de Requisitos de Software	Documento formal de requisitos.
NFR	Non-Functional Requirement	Requisito não funcional:

		qualidade/restrição.
SLA	Service Level Agreement	Acordo de nível de serviço: disponibilidade, tempo, atendimento.
CASE	Computer-Aided Software Engineering	Ferramenta de apoio à engenharia de software.
UML	Unified Modeling Language	Linguagem de modelagem; será aprofundada na Aula 2.
API	Application Programming Interface	Interface de programação para integração entre sistemas.
REST	Representational State Transfer	Estilo arquitetural para APIs; será aprofundado em arquitetura.
MoSCoW	Must, Should, Could, Won't	Priorização de requisitos.
JAD	Joint Application Development	Workshop colaborativo de elicitação e validação.

9. Pegadinhas FGV nesta aula

- **Levantamento não é tudo:** engenharia de requisitos inclui análise, especificação, validação e gerência.
- **Usuário não é único stakeholder:** auditoria, segurança, operação, gestor, sistemas externos e órgãos normativos também contam.
- **Requisito funcional não é tela:** tela é possível solução/interface.
- **Não funcional não é opcional:** desempenho, segurança e disponibilidade são requisitos de qualidade.
- **Validação não é teste unitário:** validação confirma aderência à necessidade; teste é uma forma de verificar/validar produto em outro nível.
- **Protótipo não substitui requisito:** ajuda a esclarecer, mas pode esconder regras, exceções e integrações.
- **Agile não elimina documentação:** reduz desperdício, mas mantém comunicação, rastreabilidade e critérios quando necessários.
- **Projeto não é codificação:** projeto define estrutura da solução antes/durante implementação.
- **Rastreabilidade não é burocracia inútil:** ajuda manutenção, auditoria, impacto de mudanças e cobertura de testes.

10. Mapa mental textual

Engenharia de requisitos -> elicitar -> analisar -> especificar -> validar -> gerenciar.

Elicitar -> entrevistas, workshops, observação, documentos, protótipos, questionários.

Analisar -> classificar, modelar, negociar, priorizar, verificar consistência, avaliar viabilidade.

Especificar -> ERS/SRS, casos de uso, histórias, critérios de aceite, protótipos, modelos.

Validar -> revisão com stakeholder, walkthrough, protótipo, casos de teste, matriz de rastreabilidade.

Gerenciar -> baseline, versão, mudança, impacto, rastreabilidade, priorização.

Análise de sistemas -> problema, domínio, regras, atores, dados, processos.

Projeto de sistemas -> solução, arquitetura, componentes, interfaces, banco, camadas, decisões técnicas.

11. Lista de questões autorais - sem gabarito

As questões a seguir são inéditas. Foram elaboradas com enunciados mais longos, cenários de órgão público, alternativas verossímeis e estruturas compatíveis com padrões recorrentes em cadernos FGV de TI: itens I/II/III, V/F, situações-problema e distinção entre conceitos próximos. Não há gabarito nesta seção.

1. Em um tribunal, a equipe de tecnologia iniciou o desenvolvimento de um sistema para controlar pedidos internos de suporte. Na primeira reunião, usuários relataram dificuldades, gestores apresentaram metas de redução de prazo, a equipe de segurança indicou exigências de auditoria e a área de infraestrutura apontou restrições de disponibilidade. Considerando a Engenharia de Requisitos, assinale a afirmativa correta.

- (A) A atividade de requisitos termina quando os usuários descrevem verbalmente as funcionalidades desejadas, cabendo ao projeto técnico resolver as restrições restantes.
- (B) A elicitación de requisitos consiste apenas em entrevistar usuários finais, pois gestores, áreas de controle e sistemas integrados não são fontes legítimas de requisitos.
- (C) A análise de requisitos deve evitar negociação entre stakeholders, pois conflitos devem ser resolvidos somente na etapa de codificação.
- (D) A Engenharia de Requisitos envolve descobrir, analisar, especificar, validar e gerenciar requisitos, incluindo necessidades funcionais, qualidades esperadas, restrições e rastreabilidade.
- (E) A especificação de requisitos equivale à escolha da linguagem, do framework e do banco de dados que serão usados no desenvolvimento.

2. O sistema ProcessoÁgil deverá permitir a consulta de processos por número, parte e classe processual. Além disso, deverá manter trilha de auditoria de cada acesso, responder às consultas em até dois segundos em 95% das requisições e exigir autenticação integrada ao provedor institucional de identidade. Nesse cenário, é correto classificar como requisitos não funcionais:

- (A) a trilha de auditoria, o limite de tempo de resposta e a exigência de autenticação integrada, pois expressam qualidades ou restrições do comportamento da solução.
- (B) a consulta por número, parte e classe processual, pois toda funcionalidade de pesquisa é requisito não funcional.
- (C) apenas o tempo de resposta, pois segurança e auditabilidade são regras de negócio, não requisitos de software.
- (D) todos os itens citados, pois em sistemas corporativos a diferença entre funcional e não funcional não tem utilidade para análise.
- (E) nenhum dos itens, pois requisitos não funcionais só dizem respeito à infraestrutura física, como servidores e redes.

3. Durante a modernização de um sistema legado, os usuários afirmam que “o fluxo é simples”, mas, no trabalho real, adotam exceções informais, consultam planilhas paralelas e fazem conferências manuais antes de concluir determinadas tarefas. A técnica de elicitação mais adequada para revelar esse conhecimento tácito é:

- (A) questionário fechado, pois elimina a subjetividade e captura integralmente o fluxo real de trabalho.
- (B) brainstorming com a equipe técnica, pois desenvolvedores conhecem melhor as exceções operacionais do que os usuários.
- (C) observação do trabalho ou etnografia, possivelmente combinada com entrevistas, pois permite identificar práticas reais que não aparecem no discurso formal.
- (D) análise de código-fonte, pois todo conhecimento de negócio relevante estará necessariamente implementado no sistema legado.
- (E) definição imediata da arquitetura, pois exceções operacionais devem ser tratadas apenas como detalhe de implementação.

4. A equipe produziu uma especificação de requisitos para um módulo de distribuição de tarefas. Na revisão, um analista percebeu que a expressão “o sistema deve ser rápido” era ambígua e não verificável. Em seguida, ao apresentar o protótipo ao setor demandante, constatou que o fluxo especificado não correspondia à necessidade real de redistribuição entre unidades. Nessa situação, os dois problemas identificados correspondem, respectivamente, a falhas de:

- (A) validação e implementação.
- (B) projeto e implantação.
- (C) codificação e homologação.
- (D) gerência de configuração e versionamento.
- (E) verificação do requisito e validação do requisito.

5. Joana, analista de negócios de um órgão público, está avaliando a necessidade de substituir um processo manual por uma solução informatizada. Ela identifica atores, objetivos institucionais, riscos, indicadores de desempenho, gargalos do processo atual e benefícios esperados. Essa atuação caracteriza principalmente:

- (A) projeto físico de banco de dados, pois envolve definição de entidades e relacionamentos técnicos.
- (B) análise de negócios, pois busca compreender problema, valor, contexto, stakeholders e mudança organizacional antes da solução detalhada.
- (C) codificação orientada a objetos, pois a identificação de atores equivale à implementação de classes.
- (D) teste de sistema, pois a verificação dos objetivos institucionais substitui a especificação de requisitos.
- (E) entrega contínua, pois o foco está em automatizar a implantação da solução em produção.

6. Ao revisar uma Especificação de Requisitos de Software (ERS), o coordenador técnico afirma que um bom requisito deve possuir determinadas características de qualidade. Assinale a opção que apresenta uma característica desejável de requisito bem formulado.

- (A) Ser verificável, permitindo confirmar objetivamente, por teste, inspeção, análise ou demonstração, se foi atendido.
- (B) Ser propositalmente ambíguo, para preservar liberdade interpretativa entre negócio e desenvolvimento.
- (C) Ser independente de rastreabilidade, pois rastrear requisito aumenta documentação e reduz qualidade.
- (D) Ser descrito preferencialmente em termos de tecnologia específica, mesmo quando a necessidade de negócio não exige tal restrição.
- (E) Ser incompleto na primeira versão, já que lacunas devem ser descobertas somente após a implantação.

7. Em um projeto de API institucional, a equipe mantém uma matriz ligando objetivos do órgão, requisitos, histórias de usuário, commits, casos de teste e versões implantadas. Quando um normativo muda, essa matriz ajuda a identificar funcionalidades afetadas e testes que devem ser refeitos. Essa prática é exemplo de:

- (A) refatoração, pois altera internamente o código sem modificar comportamento externo.
- (B) prototipação descartável, pois antecipa telas para obter feedback dos usuários.
- (C) encapsulamento, pois oculta detalhes internos de implementação das classes.
- (D) rastreabilidade de requisitos, pois permite acompanhar origem, implementação, testes e impacto de mudanças.
- (E) integração contínua, pois compila automaticamente o sistema após cada alteração.

8. Durante uma sprint, um desenvolvedor implementa uma tela adicional “porque seria interessante para o usuário”, embora a funcionalidade não esteja vinculada a requisito aprovado, prioridade de backlog, objetivo do produto ou solicitação validada. À luz da Engenharia de Requisitos, a situação ilustra principalmente:

- (A) validação formal, pois qualquer melhoria percebida pelo desenvolvedor aumenta aderência ao negócio.
- (B) controle adequado de escopo, pois a equipe deve incorporar toda oportunidade técnica sem análise de impacto.
- (C) gold plating ou acréscimo indevido de escopo, pois adiciona solução não solicitada e não validada.
- (D) requisito não funcional, pois telas adicionais são sempre restrições de usabilidade.
- (E) baseline de requisitos, pois toda funcionalidade implementada passa automaticamente a compor o escopo aprovado.

9. Em um projeto conduzido com práticas ágeis, a equipe registra no backlog a seguinte história: “Como servidor da unidade, quero anexar documentos ao pedido para complementar a análise”. Considerando boas práticas de especificação, assinale a afirmativa correta.

- (A) A história de usuário substitui integralmente critérios de aceite, testes e comunicação com stakeholders.
- (B) Por estar no formato “Como... quero... para...”, a história já contém todos os requisitos não funcionais associados.
- (C) Histórias de usuário não podem ser usadas em órgãos públicos, pois métodos ágeis são incompatíveis com rastreabilidade.
- (D) Uma história de usuário é sempre mais detalhada do que um caso de uso completo, incluindo pré-condições e fluxos alternativos por definição.
- (E) A história deve ser complementada por conversas, critérios de aceite e, quando necessário, regras, protótipos ou modelos que reduzam ambiguidade.

10. A equipe de um sistema de atendimento pretende reduzir incertezas sobre navegação e campos obrigatórios de uma nova funcionalidade. Para isso, constrói telas navegáveis sem integração real com o backend e as apresenta a usuários-chave. Nesse contexto, a prototipação:

- (A) é inadequada, pois protótipos somente podem ser construídos após o sistema estar implantado em produção.
- (B) apoia a elicitación e validação de requisitos, permitindo obter feedback e reduzir ambiguidades antes da implementação completa.
- (C) elimina a necessidade de especificar regras de negócio, requisitos não funcionais e integrações.
- (D) é sinônimo de teste unitário, pois verifica métodos isolados da aplicação.
- (E) deve ser usada exclusivamente para definir a estrutura física do banco de dados.

11. Em uma reunião, o product owner descreve que “apenas usuários com perfil de gestor podem aprovar solicitações acima de determinado valor”. O arquiteto propõe usar um serviço de autorização centralizado, interceptor no backend e registro de auditoria em tabela específica. A primeira frase e a proposta técnica correspondem, respectivamente, a:

- (A) requisito/regra de negócio e decisão de projeto da solução.
- (B) decisão de implantação e requisito de usabilidade.
- (C) código-fonte e requisito de domínio físico.
- (D) teste de integração e modelagem de requisitos.
- (E) baseline e eliciação por questionário.

12. Considere as afirmações sobre requisitos não funcionais em sistemas corporativos: I. Podem influenciar fortemente decisões de arquitetura, como redundância, cache, mensageria, criptografia e observabilidade. II. Tratam apenas de aparência de telas e identidade visual. III. Devem, sempre que possível, ser mensuráveis ou verificáveis. Está correto o que se afirma em:

- (A) I, apenas.
- (B) II, apenas.
- (C) I e II, apenas.
- (D) I e III, apenas.
- (E) I, II e III.

13. Na validação de requisitos de um módulo de intimações eletrônicas, a equipe reúne usuários, percorre cenários de uso, confronta fluxos alternativos, verifica protótipos e confirma se as regras documentadas representam a necessidade real. Essa atividade busca principalmente:

- (A) comprovar que cada classe Java possui cobertura mínima de testes unitários.
- (B) substituir a homologação e dispensar participação dos stakeholders até a implantação.
- (C) assegurar que os requisitos especificados correspondem ao problema e às expectativas legítimas dos stakeholders.
- (D) definir o algoritmo de ordenação mais eficiente para as consultas internas.
- (E) escolher o sistema operacional e o servidor de aplicação definitivos.

14. Após várias rodadas de negociação, um conjunto de requisitos é aprovado como referência para planejamento, desenvolvimento e testes. Duas semanas depois, uma unidade solicita alteração que impacta prazo, dados armazenados e integrações externas. De acordo com boas práticas de gerência de requisitos, a equipe deve:

- (A) recusar automaticamente qualquer alteração, pois requisitos aprovados nunca podem mudar.
- (B) implementar imediatamente a alteração, pois toda solicitação de usuário prevalece sobre prazo e arquitetura.
- (C) apagar a versão anterior dos requisitos, evitando histórico para não confundir a equipe.
- (D) tratar a mudança apenas como refatoração, sem envolvimento do negócio.
- (E) submeter a solicitação a controle de mudanças, avaliando impacto, prioridade, riscos, dependências e atualização da baseline quando aprovada.

15. Para um sistema de gestão documental do Poder Judiciário, a equipe considerou como interessados apenas os servidores que farão upload de documentos. Posteriormente, surgiram requisitos de preservação digital, metadados, auditoria, acesso externo e integração com outro sistema. O erro inicial da equipe foi:

- (A) priorizar stakeholders demais, tornando a eliciação excessivamente ampla.
- (B) adotar visão restrita de stakeholder, ignorando áreas, normas, usuários indiretos, sistemas integrados e necessidades de controle.
- (C) realizar análise de impacto antes de codificar, prática incompatível com engenharia de requisitos.
- (D) usar requisitos não funcionais, que não devem ser considerados em sistemas documentais.
- (E) separar requisito funcional de regra de negócio, distinção que não se aplica ao setor público.

16. Na priorização de requisitos de uma solução administrativa, a equipe classifica alguns itens como Must have, Should have, Could have e Won't have now, considerando valor, risco, obrigação normativa e esforço. Essa abordagem está associada à técnica:

- (A) CRUD.
- (B) MVC.
- (C) MoSCoW.
- (D) REST.
- (E) ACID.

17. Em um sistema de concessão de acesso, a norma interna determina que perfis de administrador devem ser revisados a cada 90 dias por gestor responsável. Essa determinação existe independentemente da tecnologia adotada. No contexto de requisitos, ela é melhor compreendida como:

- (A) regra de negócio ou requisito de domínio que condiciona o comportamento do sistema.
- (B) decisão física de banco de dados, pois define índices e chaves estrangeiras.
- (C) padrão de projeto, pois resolve problema recorrente de orientação a objetos.

(D) teste de regressão, pois garante que funcionalidades antigas continuem funcionando.

(E) pipeline de CI/CD, pois automatiza a entrega da aplicação.

18. Uma ferramenta CASE é adotada por uma equipe para modelar requisitos, manter diagramas, gerar documentação e apoiar rastreabilidade. Sobre o uso desse tipo de ferramenta, assinale a afirmativa correta.

(A) Ferramentas CASE substituem a elicitação junto aos stakeholders, pois inferem automaticamente todas as regras de negócio.

(B) O uso de CASE elimina a necessidade de validação, já que modelos gerados por ferramenta são sempre corretos.

(C) Ferramentas CASE são restritas à compilação de código e não se relacionam com modelagem ou documentação.

(D) Ferramentas CASE apoiam atividades de engenharia de software, mas não substituem análise crítica, negociação e validação de requisitos.

(E) Ferramentas CASE são incompatíveis com UML e com documentação técnica.

19. Com relação ao ciclo de vida e aos requisitos, assinale (V) para a afirmativa verdadeira e (F) para a falsa. () Em modelos sequenciais, como o cascata, há tendência de maior formalização e estabilização de requisitos antes de fases posteriores, embora mudanças possam ocorrer mediante controle. () Em abordagens iterativas e ágeis, requisitos podem ser refinados ao longo do projeto, mas isso não significa ausência de priorização, validação ou critérios de aceite. () A análise de requisitos deve ser eliminada quando se usa Scrum, pois o backlog substitui qualquer necessidade de entendimento do problema. As afirmativas são, respectivamente:

(A) F, F e V.

(B) V, F e V.

(C) F, V e F.

(D) V, V e V.

(E) V, V e F.

20. Um analista escreveu o requisito “o sistema deverá usar PostgreSQL para armazenar os dados de solicitações”. Ao revisar o documento, outro analista percebe que a necessidade real era “armazenar solicitações com integridade, disponibilidade e possibilidade de auditoria”. A observação correta é que:

(A) a primeira redação é sempre superior, pois requisitos devem privilegiar tecnologia em vez de necessidade.

(B) a primeira redação pode ser uma restrição de solução, mas não expressa, por si só, a necessidade de negócio que justifica o armazenamento.

(C) a segunda redação é inválida, pois requisitos não podem mencionar qualidades como integridade e auditabilidade.

(D) ambas as redações são casos de uso completos, com ator, fluxo principal e pós-condições.

(E) a escolha de banco de dados é sempre requisito funcional obrigatório definido pelo usuário final.

12. Gabarito resumido

1-D, 2-A, 3-C, 4-E, 5-B, 6-A, 7-D, 8-C, 9-E, 10-B, 11-A, 12-D, 13-C, 14-E, 15-B, 16-C, 17-A, 18-D, 19-E, 20-B

Distribuição do gabarito

Aula com distribuição equilibrada: 4 alternativas A, 4 B, 4 C, 4 D e 4 E. Essa distribuição foi usada para treino e não como afirmação estatística sobre a FGV.

13. Questões comentadas

Questão 1 - Gabarito: D

A alternativa D está correta porque engenharia de requisitos é um processo amplo: envolve descoberta, análise, documentação, validação e controle ao longo do ciclo de vida. Em sistemas corporativos, inclui requisitos funcionais, não funcionais, regras, restrições e rastreabilidade.

(A) Errada. Usuários relatam necessidades, mas isso não encerra requisitos; restrições de segurança, auditoria e disponibilidade também devem ser tratadas.

(B) Errada. Usuário final é stakeholder importante, mas não é a única fonte. Gestores, normas, sistemas externos e áreas de controle também geram requisitos.

(C) Errada. Conflitos devem ser analisados e negociados antes de virarem defeitos de escopo ou retrabalho.

(D) **Correta.** A Engenharia de Requisitos envolve descobrir, analisar, especificar, validar e gerenciar requisitos, incluindo necessidades funcionais, qualidades esperadas, restrições e rastreabilidade.

(E) Errada. Linguagem, framework e banco são decisões/restrições de projeto, não equivalem à especificação completa de requisitos.

Erro provável

Confusão entre “pedido do usuário” e processo completo de engenharia de requisitos.

Questão 2 - Gabarito: A

A alternativa A está correta. Auditoria, desempenho e autenticação integrada descrevem qualidades ou restrições do sistema, não apenas funções de negócio. Embora autenticação tenha comportamento funcional, a exigência de integração institucional e controle de acesso aparece como requisito de segurança/restrição de qualidade no cenário.

(A) **Correta.** a trilha de auditoria, o limite de tempo de resposta e a exigência de autenticação integrada, pois expressam qualidades ou restrições do comportamento da solução.

(B) Errada. Consultar por número, parte e classe é funcionalidade oferecida pelo sistema.

(C) Errada. Segurança e auditabilidade podem ser requisitos não funcionais essenciais.

(D) Errada. A distinção continua útil para análise, arquitetura e teste.

(E) Errada. Requisitos não funcionais não se limitam a infraestrutura física.

Erro provável

Memorização fraca da diferença entre funcional e não funcional.

Questão 3 - Gabarito: C

A alternativa C está correta. Observação/etnografia é adequada quando há diferença entre processo declarado e processo real, especialmente quando usuários executam passos tácitos ou informais.

(A) Errada. Questionário fechado alcança muitos usuários, mas tem pouca profundidade para exceções informais.

(B) Errada. Desenvolvedores podem apoiar análise, mas não substituem observação do trabalho real.

(C) **Correta.** observação do trabalho ou etnografia, possivelmente combinada com entrevistas, pois permite identificar práticas reais que não aparecem no discurso formal.

(D) Errada. Código legado pode conter regras, mas nem todo conhecimento operacional está nele.

(E) Errada. Arquitetura sem compreensão das exceções tende a cristalizar erro de entendimento.

Erro provável

Escolha mecânica de técnica de elicitação sem observar o tipo de conhecimento buscado.

Questão 4 - Gabarito: E

A alternativa E está correta. “O sistema deve ser rápido” é problema de verificação do requisito: está ambíguo e não testável. O protótipo não refletir a necessidade real é problema de validação: o requisito certo não foi capturado.

(A) Errada. A primeira falha não é validação; é qualidade interna do requisito.

(B) Errada. Não se trata ainda de projeto/implantação.

(C) Errada. Não há menção a código nem homologação como fase de teste do produto.

(D) Errada. Gerência de configuração e versionamento não são o centro do problema.

(E) **Correta.** verificação do requisito e validação do requisito.

Erro provável

Confusão clássica entre verificação e validação.

Questão 5 - Gabarito: B

A alternativa B está correta. O foco descrito é entender problema, valor, stakeholders, riscos e mudança organizacional antes de detalhar a solução. Isso é típico de análise de negócios.

(A) Errada. Não há definição física de banco de dados.

(B) **Correta.** análise de negócios, pois busca compreender problema, valor, contexto, stakeholders e mudança organizacional antes da solução detalhada.

(C) Errada. Atores em análise não equivalem diretamente a classes implementadas.

(D) Errada. Teste de sistema não substitui levantamento e análise.

(E) Errada. Entrega contínua trata automação de entrega, não análise de valor e processo.

Erro provável

Confundir análise de negócios com análise técnica ou desenvolvimento.

Questão 6 - Gabarito: A

A alternativa A está correta. Requisito verificável permite confirmar se foi atendido por teste, inspeção, análise ou demonstração. Essa é característica central de requisito bem escrito.

(A) **Correta.** Ser verificável, permitindo confirmar objetivamente, por teste, inspeção, análise ou demonstração, se foi atendido.

(B) Errada. Ambiguidade aumenta conflito e retrabalho.

(C) Errada. Rastreabilidade é desejável, principalmente em sistemas auditáveis.

(D) Errada. Tecnologia específica só deve aparecer quando for restrição real.

(E) Errada. Lacunas devem ser tratadas o quanto antes; podem evoluir, mas não são qualidade.

Erro provável

Aceitar redações vagas como se fossem flexibilidade saudável.

Questão 7 - Gabarito: D

A alternativa D está correta. A matriz descrita liga origem, requisito, implementação, teste e release; isso é rastreabilidade e apoia análise de impacto.

(A) Errada. Refatoração altera estrutura interna do código sem mudar comportamento externo.

(B) Errada. Prototipação envolve modelos/telas para feedback.

(C) Errada. Encapsulamento é princípio de orientação a objetos.

(D) **Correta.** rastreabilidade de requisitos, pois permite acompanhar origem, implementação, testes e impacto de mudanças.

(E) Errada. Integração contínua automatiza build/testes, mas não descreve matriz de origem-impacto.

Erro provável

Confusão entre práticas de engenharia de requisitos e práticas de desenvolvimento.

Questão 8 - Gabarito: C

A alternativa C está correta. Implementar funcionalidade não solicitada, sem validação ou vínculo a escopo, é gold plating: acréscimo indevido que pode gerar custo, risco e manutenção desnecessária.

(A) Errada. Validação envolve confirmar necessidade com stakeholders.

(B) Errada. Controle de escopo exige análise de impacto e aprovação.

(C) **Correta.** gold plating ou acréscimo indevido de escopo, pois adiciona solução não solicitada e não validada.

(D) Errada. Tela adicional não é automaticamente requisito não funcional.

(E) Errada. Implementar algo não o torna baseline aprovada.

Erro provável

Achar que toda melhoria técnica agrega valor automaticamente.

Questão 9 - Gabarito: E

A alternativa E está correta. História de usuário é artefato leve e deve ser complementada por conversas, critérios de aceite e detalhes necessários ao contexto.

- (A) Errada. História não substitui critérios de aceite e comunicação.
- (B) Errada. O formato não garante requisitos não funcionais.
- (C) Errada. Métodos ágeis podem ser usados no setor público com governança adequada.
- (D) Errada. Caso de uso completo tende a ser mais estruturado que uma história.
- (E) **Correta.** A história deve ser complementada por conversas, critérios de aceite e, quando necessário, regras, protótipos ou modelos que reduzam ambiguidade.

Erro provável

Romantizar artefato ágil e ignorar critérios verificáveis.

Questão 10 - Gabarito: B

A alternativa B está correta. Protótipos ajudam a esclarecer requisitos, validar fluxo e reduzir ambiguidade antes da implementação completa.

- (A) Errada. Protótipos são especialmente úteis antes da construção final.
- (B) **Correta.** apoia a elicitação e validação de requisitos, permitindo obter feedback e reduzir ambiguidades antes da implementação completa.
- (C) Errada. Protótipo não elimina regra, NFR e integração.
- (D) Errada. Teste unitário verifica unidades de código.
- (E) Errada. Prototipação de interface não é exclusiva de banco de dados.

Erro provável

Confundir protótipo com produto final ou com teste de código.

Questão 11 - Gabarito: A

A alternativa A está correta. A regra de aprovação por perfil/alçada pertence ao domínio do negócio; interceptor, serviço e tabela são decisões de projeto para implementar a regra.

- (A) **Correta.** requisito/regra de negócio e decisão de projeto da solução.
- (B) Errada. Não é implantação nem requisito de usabilidade.
- (C) Errada. A primeira frase não é código-fonte.
- (D) Errada. Não descreve teste de integração.
- (E) Errada. Não há baseline nem questionário.

Erro provável

Misturar regra de negócio com solução técnica.

Questão 12 - Gabarito: D

A alternativa D está correta. Requisitos não funcionais orientam arquitetura e devem ser mensuráveis ou verificáveis sempre que possível. A afirmativa II é falsa porque NFRs vão muito além de aparência de tela.

- (A) Errada. Ignora a afirmativa III, que também está correta.
- (B) Errada. Afirmativa II é falsa.
- (C) Errada. Inclui II, falsa, e exclui III, verdadeira.
- (D) **Correta.** I e III, apenas.
- (E) Errada. Inclui II, falsa.

Erro provável

Reduzir requisito não funcional a interface visual.

Questão 13 - Gabarito: C

A alternativa C está correta. Validar requisitos é confirmar se eles representam a necessidade real dos stakeholders e do problema.

- (A) Errada. Cobertura de classe Java é teste unitário/qualidade de código.

- (B) Errada. Validação exige participação dos interessados, não dispensa.
- (C) **Correta.** assegurar que os requisitos especificados correspondem ao problema e às expectativas legítimas dos stakeholders.
- (D) Errada. Algoritmo de ordenação é decisão técnica específica.
- (E) Errada. Escolhas de SO/servidor são projeto/infraestrutura.

Erro provável

Confundir validação de requisitos com testes técnicos da implementação.

Questão 14 - Gabarito: E

A alternativa E está correta. Baseline não congela o mundo; ela cria referência. Mudanças devem passar por controle, análise de impacto e atualização quando aprovadas.

- (A) Errada. Requisitos podem mudar mediante controle.
- (B) Errada. Solicitação deve ser analisada, não implementada automaticamente.
- (C) Errada. Histórico é importante para rastreabilidade.
- (D) Errada. Mudança de requisito não é apenas refatoração.
- (E) **Correta.** submeter a solicitação a controle de mudanças, avaliando impacto, prioridade, riscos, dependências e atualização da baseline quando aprovada.

Erro provável

Pensar em baseline como proibição absoluta de mudança.

Questão 15 - Gabarito: B

A alternativa B está correta. Stakeholders incluem usuários diretos, indiretos, áreas normativas, segurança, auditoria, gestão documental, sistemas integrados e órgãos reguladores quando influenciam requisitos.

- (A) Errada. O problema foi considerar stakeholders de menos.
- (B) **Correta.** adotar visão restrita de stakeholder, ignorando áreas, normas, usuários indiretos, sistemas integrados e necessidades de controle.
- (C) Errada. Análise de impacto é prática adequada.
- (D) Errada. Sistemas documentais têm fortes requisitos não funcionais.
- (E) Errada. A distinção continua aplicável no setor público.

Erro provável

Assumir que stakeholder é apenas quem clica na tela.

Questão 16 - Gabarito: C

A alternativa C está correta. MoSCoW significa Must, Should, Could e Won't, técnica de priorização de requisitos.

- (A) Errada. CRUD é Create, Read, Update, Delete.
- (B) Errada. MVC é Model-View-Controller.
- (C) **Correta.** MoSCoW.
- (D) Errada. REST é estilo arquitetural.
- (E) Errada. ACID é conjunto de propriedades transacionais.

Erro provável

Siglas próximas do cotidiano técnico confundindo técnica de requisitos.

Questão 17 - Gabarito: A

A alternativa A está correta. A revisão periódica de perfis vem de norma/domínio e condiciona o comportamento do sistema; não depende da tecnologia escolhida.

- (A) **Correta.** regra de negócio ou requisito de domínio que condiciona o comportamento do sistema.
- (B) Errada. Não define índice ou chave estrangeira.
- (C) Errada. Padrão de projeto resolve problema de desenho de software.
- (D) Errada. Teste de regressão é atividade de teste.
- (E) Errada. CI/CD automatiza integração e entrega.

Erro provável

Não reconhecer regra normativa como requisito/regra de negócio.

Questão 18 - Gabarito: D

A alternativa D está correta. CASE apoia modelagem, documentação, rastreabilidade e outras atividades, mas não substitui análise, negociação e validação.

(A) Errada. Ferramenta não descobre automaticamente todas as regras.

(B) Errada. Modelos precisam ser validados.

(C) Errada. CASE não se restringe à compilação.

(D) **Correta.** Ferramentas CASE apoiam atividades de engenharia de software, mas não substituem análise crítica, negociação e validação de requisitos.

(E) Errada. CASE pode apoiar UML e documentação.

Erro provável

Atribuir poder mágico à ferramenta ou subestimar sua função de apoio.

Questão 19 - Gabarito: E

A alternativa E está correta: V, V e F. Modelos sequenciais tendem a maior formalização inicial; abordagens iterativas/ágeis refinam requisitos, mas não eliminam disciplina; Scrum não dispensa entendimento do problema.

(A) Errada. As duas primeiras são verdadeiras.

(B) Errada. A segunda é verdadeira e a terceira é falsa.

(C) Errada. A primeira é verdadeira.

(D) Errada. A terceira é falsa.

(E) **Correta.** V, V e F.

Erro provável

Confundir agilidade com ausência de análise e documentação útil.

Questão 20 - Gabarito: B

A alternativa B está correta. “Usar PostgreSQL” pode ser restrição de solução se houver justificativa institucional, mas não expressa a necessidade essencial de armazenar com integridade, disponibilidade e auditoria.

(A) Errada. Requisitos devem partir da necessidade; tecnologia só entra como restrição justificada.

(B) **Correta.** a primeira redação pode ser uma restrição de solução, mas não expressa, por si só, a necessidade de negócio que justifica o armazenamento.

(C) Errada. Integridade e auditabilidade são qualidades relevantes.

(D) Errada. Nenhuma das redações é caso de uso completo.

(E) Errada. Banco específico não é sempre requisito funcional do usuário.

Erro provável

Confundir necessidade de negócio com decisão tecnológica.

14. Flashcards finais

Pergunta	Resposta
O que é Engenharia de Requisitos?	Processo de descobrir, analisar, especificar, validar e gerenciar requisitos.
Elicitação é só entrevista?	Não. Inclui entrevistas, workshops, observação, documentos, questionários, protótipos etc.
Requisito funcional?	Função ou comportamento que o sistema deve oferecer.
Requisito não funcional?	Qualidade, propriedade ou restrição: segurança, desempenho, disponibilidade, usabilidade etc.
Regra de negócio?	Regra do domínio/organização/legislação que condiciona o sistema.
Requisito bom?	Necessário, claro, completo, consistente, verificável, viável, rastreável e priorizado.
Verificação de requisito?	Confere se o requisito está bem escrito e consistente.

Validação de requisito?	Confirma se o requisito representa a necessidade real.
Rastreabilidade?	Ligação entre origem, requisito, implementação, teste e entrega.
Baseline?	Versão aprovada de requisitos usada como referência e controlada por mudança.
Análise x projeto?	Análise entende o problema; projeto desenha a solução.
CASE?	Computer-Aided Software Engineering: ferramenta que apoia, mas não substitui análise.
História de usuário substitui documentação?	Não. Precisa de conversa, critérios de aceite e detalhes quando necessários.
Protótipo substitui requisito?	Não. Ele ajuda a esclarecer e validar, mas não cobre tudo.
Stakeholder é só usuário final?	Não. Inclui todos que afetam ou são afetados pela solução.

15. Checklist de memorização

- Decorar a sequência: elicitar -> analisar -> especificar -> validar -> gerenciar.
- Saber diferenciar funcional, não funcional, regra de negócio, requisito de domínio e restrição de projeto.
- Associar requisito bom a verificável, não ambíguo e rastreável.
- Separar verificação (“bem escrito?”) de validação (“é o que precisa?”).
- Entender rastreabilidade como ferramenta de impacto, manutenção, auditoria e teste.
- Separar análise (“o quê/por quê”) de projeto (“como”).
- Lembrar que ágil não elimina análise, critérios de aceite nem controle de mudança.
- Lembrar que protótipo ajuda, mas não substitui requisitos completos.
- Em questões FGV, procurar palavras absolutas: sempre, nunca, apenas, automaticamente, exclusivamente.

16. Revisão ativa em 30 minutos

1. 5 minutos: releia o mapa mental textual e as pegadinhas.
2. 10 minutos: responda novamente às questões 1, 4, 7, 11, 14 e 20 sem olhar gabarito.
3. 10 minutos: transforme os erros em flashcards no caderno de erros.
4. 5 minutos: escreva, de memória, a diferença entre requisito funcional, não funcional, regra de negócio e restrição de projeto.

17. Referências consultadas

As referências abaixo foram usadas para alinhamento técnico, estrutura conceitual e observação do padrão de enunciados da FGV. O material é autoral e não copia questões reais.

- FGV Conhecimento - Edital Retificado TJSC Servidor 2026: distribuição de questões e conteúdo programático de Analista de Sistemas.
- FGV Conhecimento - Cadernos oficiais recentes de Analista de Sistemas/Desenvolvimento de Sistemas: AL-GO 2026, ALESC 2024, MPU Desenvolvimento de Sistemas e TJ-MT Analista de Sistemas Computacionais.
- ISO/IEC/IEEE 29148:2018 - Systems and software engineering - Life cycle processes - Requirements engineering.
- IEEE/ISO/IEC 29148 - descrição do constructo de bom requisito, atributos e aplicação iterativa dos processos de requisitos.
- IIBA BABOK Guide - Requirements Analysis and Design Definition; Requirements Life Cycle Management.
- OMG - UML 2.5.1 Specification, como referência de linguagem de modelagem para artefatos de sistemas.
- Sommerville, Ian. Engenharia de Software. Conceitos de requisitos, processos, validação e projeto.
- Pressman, Roger; Maxim, Bruce. Engenharia de Software. Requisitos, modelagem e projeto.